

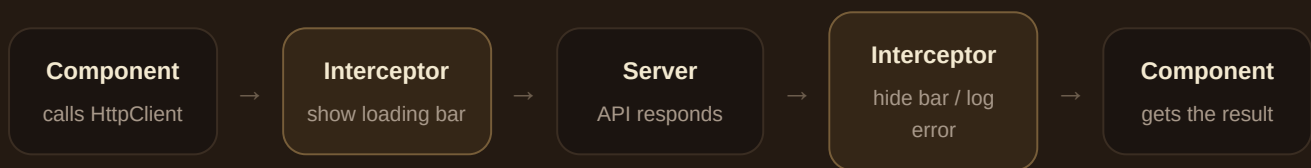
HttpInterceptor

One checkpoint that **every outgoing HTTP request and incoming response** passes through — think of it as airport security for HTTP calls: one place for loading state, auth headers, or error logging.

1 · EXPLAINED SIMPLY

Every request passes through the same checkpoint

An interceptor sits between your code and the network. It can inspect or modify the outgoing request, and inspect or modify the response (or error) on the way back — for every single HTTP call, without touching each component individually.



2 · THE CODE

A functional `HttpInterceptorFn`

```
export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loadingService = inject(LoadingService);
  loadingService.show();

  return next(req).pipe(
    catchError((error) => {
      console.error(`Request to ${req.url} failed:`, error.message);
      return throwError(() => error);
    }),
    finalize(() => loadingService.hide())
  );
};
```

- `show()` / `hide()` just increment / decrement a counter signal — `LoadingService.isLoading()` is true while it's above zero.
- `finalize()` runs whether the request succeeds, fails, or is cancelled by `switchMap` — the loading bar never gets stuck on.
- Registered once: `provideHttpClient(withInterceptors([loadingInterceptor]))` in `app.config.ts`.

`src/app/core/interceptors/loading.interceptor.ts`